L18 — Insertion and Deletion in a Linked List

Unit: 2 | **Chapter:** Linked Lists | **BT Level:** BT3 | **CO:** CO2

Learning Objectives

- Implement insertion at the beginning, end, and a given position in a singly linked list
- Implement deletion from the beginning, end, and by value in a singly linked list
- Analyze time and space complexity for each operation
- Handle edge cases (empty list, single element, invalid position)

---

1. Insertion in a Singly Linked List

1.1 Insert at the Beginning — O(1)

```
def insert_at_beginning(self, data):
    new_node = Node(data)
    new_node.next = self.head   # New node points to current head
    self.head = new_node        # Update head to new node
```

Before: HEAD → [20] → [30] → NULL
After: HEAD → [10] → [20] → [30] → NULL

Steps:

1. Create new node with data = 10
2. Set `new_node.next = HEAD` (link to old head)
3. Set `HEAD = new_node` (update head)

No shifting needed! This is O(1) — the key advantage over arrays.

1.2 Insert at the End — O(n)

```
def insert_at_end(self, data):
    new_node = Node(data)
    if self.head is None:       # Empty list
        self.head = new_node
        return
    current = self.head
    while current.next:         # Traverse to last node
        current = current.next
    current.next = new_node     # Last node now points to new node
```

O(n) because we must traverse to find the last node. O(1) with a tail pointer.

1.3 Insert After a Given Node — O(1) after finding the node

```
def insert_after(self, prev_node, data):
    """Insert a new node after prev_node."""
    if prev_node is None:
        raise ValueError("Previous node cannot be None")
    new_node = Node(data)
    new_node.next = prev_node.next   # New node links to what came afte
    prev_node.next = new_node        # Prev node now links to new node
```

Trace:
Before: ... → [20] → [40] → ...
Insert 30 after node containing 20:
After: ... → [20] → [30] → [40] → ...

1.4 Insert at Position k — O(k)

```
def insert_at_position(self, pos, data):
    """Insert at position pos (0-based). pos=0 inserts at head."""
    if pos == 0:
        self.insert_at_beginning(data)
        return

    new_node = Node(data)
    current = self.head
    for i in range(pos - 1):    # Traverse to node before position
        if current is None:
            raise IndexError("Position out of range")
        current = current.next

    if current is None:
        raise IndexError("Position out of range")

    new_node.next = current.next
    current.next = new_node
```

---

2. Deletion from a Singly Linked List

2.1 Delete from the Beginning — O(1)

```
def delete_from_beginning(self):
    if self.head is None:
        raise Exception("List is empty")
    deleted_data = self.head.data
    self.head = self.head.next   # Advance head to next node
    return deleted_data
```

Before: HEAD → [10] → [20] → [30] → NULL
After: HEAD → [20] → [30] → NULL

2.2 Delete from the End — O(n)

```
def delete_from_end(self):
    if self.head is None:
        raise Exception("List is empty")
    if self.head.next is None:    # Single element
        deleted = self.head.data
        self.head = None
        return deleted

    current = self.head
    while current.next.next:      # Stop at second-to-last
        current = current.next
    deleted = current.next.data
    current.next = None           # Unlink last node
    return deleted
```

2.3 Delete a Node by Value — O(n)

```
def delete_by_value(self, key):
    """Delete first occurrence of key."""
    if self.head is None:
        return False

    # Special case: head contains the key
    if self.head.data == key:
        self.head = self.head.next
        return True

    # Find the node BEFORE the one to delete
    current = self.head
    while current.next:
        if current.next.data == key:
            current.next = current.next.next   # Bypass the node
            return True
        current = current.next

    return False   # Key not found
```

Trace — Delete 20 from [10 → 20 → 30 → NULL]:

1. `current = 10`, `current.next.data == 20` → True
2. `current.next = current.next.next` = 30's node
3. Result: `10 → 30 → NULL`

2.4 Delete at Position k — O(k)

```
def delete_at_position(self, pos):
    if self.head is None:
        raise Exception("List is empty")
```

```
    if pos == 0:
        return self.delete_from_beginning()

    current = self.head
    for i in range(pos - 1):
        if current.next is None:
            raise IndexError("Position out of range")
        current = current.next

    if current.next is None:
        raise IndexError("Position out of range")

    deleted = current.next.data
    current.next = current.next.next
    return deleted
```

---

3. Complete Linked List Implementation

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def insert_end(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        curr = self.head
        while curr.next:
            curr = curr.next
        curr.next = new_node

    def delete_value(self, key):
        if not self.head:
            return
        if self.head.data == key:
            self.head = self.head.next
            return
        curr = self.head
        while curr.next:
            if curr.next.data == key:
```

```
                curr.next = curr.next.next
                return
            curr = curr.next

    def display(self):
        curr = self.head
        result = []
        while curr:
            result.append(str(curr.data))
            curr = curr.next
        print(" → ".join(result) + " → NULL")


# Demo
ll = LinkedList()
ll.insert_end(10)
ll.insert_end(20)
ll.insert_end(30)
ll.insert_beginning(5)
ll.display()          # 5 → 10 → 20 → 30 → NULL

ll.delete_value(20)
ll.display()          # 5 → 10 → 30 → NULL
```

---

4. Complexity Summary

Operation	Time	Space
Insert at beginning	O(1)	O(1)
Insert at end	O(n) [O(1) with tail]	O(1)
Insert at position k	O(k)	O(1)
Delete from beginning	O(1)	O(1)
Delete from end	O(n)	O(1)
Delete by value	O(n)	O(1)
Delete at position k	O(k)	O(1)

Summary

- **Insertion at beginning** is O(1) — just update HEAD (faster than arrays!).
- **Deletion from beginning** is O(1) — just advance HEAD.
- Insertion/deletion **at end or by position** require O(n) traversal.
- The critical pattern: to delete node `curr.next`, set `curr.next = curr.next.next`.
- Maintaining a **tail pointer** reduces end-insertion to O(1).

References

- Cormen et al., *Introduction to Algorithms*, Ch. 10.2 (MIT Press, 2022)

- Skiena, *The Algorithm Design Manual*, Ch. 3.1 (Springer, 2020)